

Task 1

1. Display both encrypted images
 - a. AES-256-CBC Encryption
 - i. K =
123456789012345600
 - ii. iv=12345678901234567890123456789012



b. AES-256-ECB Encryption:

i. K =

123456789012345600



2. Explain your observations

- a. The ECB encrypted image is visually recognizable by its patterns and frequency of colours, while the CBC encryption is visually unrecognizable.

3.

- a. The ECB encryption is simple to decrypt visually, due to the property of ECB encryption producing identical ciphertext blocks for identical plaintext blocks. In this case, blocks containing solely the greyish pixels of Squidward's skin consistently turn into blocks containing a predictable pattern of blue, black, and white (Close up picture added below). Blocks that contain a less consistent variety of pixels (Such as a block with 40% squidward's skin and 60% squidward's black outline) produce an output that looks more random due to the infrequency of these blocks. By doing sort of a visual frequency analysis and finding patterns in the encrypted image, the shape of the plain image can be deduced, in this case the pattern of Squidward's character.



- i.
- b. The CBC encryption is unrecognizable visually, and on its own does not provide any substance for a frequency analysis. Since the random IV alters the first block in an unpredictable way (to those who don't know the IV), and the ciphertext of each block persists on altering its following block, the unpredictable change of the IV cascades and affects each block of the file unpredictably, leaving an unrecognizable image.

Task 2

1. ECB, CBC, CFB, OFB, and CTR to encrypt a file using 128-bit AES. Report which modes have padding and which ones do not. For those that do not need padding, explain why.
 - a. Uses Padding
 - i. ECB, CBC
 - b. Doesn't use Padding
 - i. CFB, OFB, CTR
 - ii. For each of these encryption methods that don't use padding, they do not need to add padding to the plaintext as the size of the plaintext does not need to be a multiple of the block size. In these encryption methods the plaintext is xor'd AFTER the block cipher encryption, meaning the final piece of plaintext can be smaller than blocksize.
2. Describe file sizes
 - a. 5.txt has a length of 5 bytes, while 5.bin has a length of 16 bytes
 - b. 10.txt has a length of 10 bytes, while 10.bin has a length of 16 bytes
 - c. 16.txt has a length of 16 bytes, while 16.bin has a length of 32 bytes
3. Identify the padding
 - a. 5pad.txt has a length of 16, the last 11 bytes are the hexadecimal value 0b (11 in decimal)

- b. 10pad.txt has a length of 16, the last 6 bytes are the hexadecimal value 06 (6 in decimal)
 - c. 16pad.txt has a length of 32, the last 16 bytes are the hexadecimal value 10 (16 in decimal)
- 4. Does 256-bit AES instead of 128-bit impact padding
 - a. No, this does not impact the padding of the files. The padding remains the same as the difference between 128 and 256-bit AES is the key size, while the block size is always 16 bytes.

Task 3

1. How much plaintext can be recovered if a bit in the 10th byte is flipped?
 - a. ECB - Only the block that processes the corrupted cypher text will be affected. Since ECB does not chain its blocks together in any way, the bit flip will have a localized effect, heavily affecting just the single block.
 - b. CBC - The majority of the plaintext will be recovered. Only two of the blocks will be affected, that being the block containing the bit flip and the one after. Since CBC uses ciphertext block N before Block Cipher Decryption for plaintext block N, but after Block Cipher Decryption for plaintext N+1, the first block affected will be very scrambled while the second block will only have a single bit flipped.
 - c. CFB - Similar to CBC, ciphertext N is used for plaintext blocks N and N+1, but Block Cipher Decryption is used before the ciphertext on block N and after the ciphertext on block N+1. This would mean the opposite of CBC, where plaintext block N would be heavily altered but plaintext block N+1 would only have a bit flipped.
 - d. OFB - Only the corrupted block will be affected, and this affect will reflect the flipped bit. OFB does not link its ciphertexts, and the ciphertext is xor'd after the Block Cipher Decryption, so the resulting plaintext will only have a bit flip where the bit flip occurred in the ciphertext.

Task 4

1. Compare same IV and unique ID ciphertexts
 - a. The same message encrypted with identical IVs produces identical encrypted texts, while having different IVs produces very different encrypted texts.
2. Reusing IV but with different texts
 - a. Since OFB does not chain its ciphertexts together, positional changes in plaintexts will result in the same positional changes in cyphertexts. In the case of P2, I can see that the first half of C2 matches C1, meaning that the first half of P2 is different from P1. From here, I know the second half of P1 "a known message" (roughly) is the end of P2, and I could guess the start of P2 is "This isn't" or something similar that follows English grammar.
 - b. Should the method switch to CFB, this would help us a bit in the given scenario, but not in all cases. Since in CFB the encryption of a plaintext block is chained to the encryption of all proceeding plaintext blocks, the ciphertext will only change once a change is made in plaintext. For instance, if P2 is something like "This isn't a known message", the ciphertext for C1 and C2 will start the same when they cover "This is", but will diverge from thereon since the plaintexts diverge there.

3. Guessing Bob's plaintext

- a. While Bob's plaintext is hidden, doing some arithmetic and encryption with the IVs and "Yes" or "No" can reveal Bob's plaintext, despite the key being hidden. Since we know Bob's IV, ciphertext, IV_{next} , and Bob kindly uses his cipher algorithm for us, we can test the cipher algorithm to try and create the same ciphertext that Bob uses, which we can do knowing the upcoming IV.
 - i. The first ciphertext block produced, C_1 , can be written as a function: $C_1 = AES_k(P_1 \text{ xor } IV_{Bob})$, where AES is the encryption function using key K, P_1 is the first block of Bob's "Yes" or "No", and IV_{Bob} is the IV we are provided initially. Now, we cannot break the AES_k off on its own to get P_1 , but since we know the upcoming IV_{next} and the value of $AES_k(P' \text{ xor } IV_{next})$, we can manipulate P' (upcoming plaintext) to try and produce a C_2 that is identical to C_1 .
 1. $C_1 = C_2$
 2. $AES_k(P_1 \text{ xor } IV_{Bob}) = AES_k(P' \text{ xor } IV_{next})$
 - a. $AES_k()$ is deterministic and constant thus can be removed
 3. $P_1 \text{ xor } IV_{Bob} = P' \text{ xor } IV_{next}$
 4. $P_1 \text{ xor } IV_{Bob} \text{ xor } IV_{next} = P' \text{ xor } IV_{next} \text{ xor } IV_{next}$
 5. $P_1 \text{ xor } IV_{Bob} \text{ xor } IV_{next} = P'$
 - a. IV_{Bob} IV_{next} are known, and P_1 is either "Yes" or "No", so two values of P' can be created.
 - ii. One of the P' values ("Yes", for instance) can now be sent into the oracle, if the beginning of the result matches Bob's original C_1 then Bob's plaintext will have been "Yes", and if it does not match then Bob's plaintext will have been "No". The reason only the beginning will match is padding will likely need to be added to the plaintext, resulting in diverging lengths as P' may not be the same length as P_1

Task 5

1. The key used was "seveneight#####" and the plaintext was "Why don't jokes work in octal? Because 7 10 11."
2. .c file included in the submission (jeez this problem took a while!)

```
[09/17/25] seed@VM:~/.../task5$ gcc -o myenc myenc.c -lcrypto
[09/17/25] seed@VM:~/.../task5$ ./myenc
Found a word!
Key (hex): 736576656e65696768742323232323
Plaintext: Why don't jokes work in octal? Because 7 10 11.
Time taken: 736 seconds
```